

系统开发工具基础

第 4 周实验报告

姓名：朱元清 学号：25020007193

2026 年 9 月 14 日

本报告记录第 4 周代码质量练习与课堂综合题的实际执行结果。内容覆盖格式化、静态检查、测试、覆盖率、提交前检查、持续集成、命令运行器、正则表达式，以及 q13-q16 的质量门禁、Make 构建、API 报告和软件包交付。

表 1: 第 4 周任务完成情况

编号	主题	关键工具	状态
13	建立可执行的本地质量门禁	Ruff、pytest、Shell	完成
14	让 Make 只重建受影响的产物	Make、Python	完成
15	把本地 API 数据转换为可读报告	curl、jq、Markdown	完成
16	修复并交付陌生小型工具包	Make、build、Git	完成
1	格式化与只检查模式	Ruff format	完成
2	静态检查与自动修复	Ruff check	完成
3	单元测试与边界条件	pytest	完成
4	覆盖率报告与补测	coverage.py、pytest	完成
5	提交前质量门禁	pre-commit	完成
6	持续集成工作流	GitHub Actions	完成
7	统一命令入口	Make	完成
8	查找危险 shell 调用	grep、regex	完成
9	Markdown 精确替换	regex	完成
10	JSON 名称提取	regex、json	完成

练习 13 建立可执行的本地质量门禁（Ruff、pytest 与 Shell）

执行内容

以 q10 的 greetlab 项目为基础，在 q13 中保留 `src/` 与 `tests/` 的标准结构，补充 Ruff 和 pytest 配置，并创建 `check.sh`。脚本依次执行格式检查、静态检查和单元测试；空白姓名应触发 `argparse` 错误并返回退出码 2。

```
python -m ruff format .
python -m ruff check .
python -m pytest
./check.sh
echo $?
```



```
(.venv)kali@kali: ~/Desktop/新建文件夹/q13
with patch.object(sys, "argv", ["sdt-greet", "--name", " "]):
    with self.assertRaises(SystemExit) as caught:
        main()

    self.assertEqual(caught.exception.code, 2)

if __name__ == "__main__":
    unittest.main()
EOF

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
$ ls
check.sh  pyproject.toml  src  tests

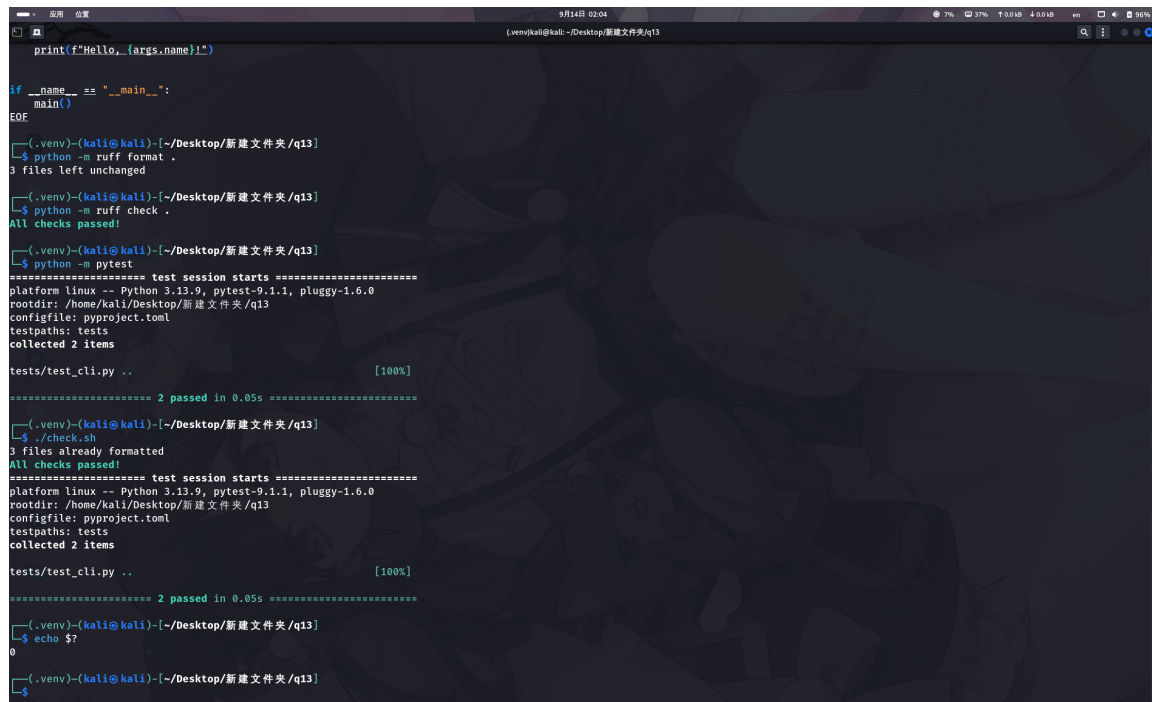
(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
$ vim pyproject.toml

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
$ cat > check.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

python -m ruff format --check .
python -m ruff check .
python -m pytest
EOF

chmod +x check.sh
```

图 1: q13 的测试目录与检查脚本配置



```
print(f"Hello, {args.name}!")

if __name__ == "__main__":
    main()
EOF

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$ python -m ruff format .
3 files left unchanged

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$ python -m ruff check .
All checks passed!

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$ python -m pytest
===== test session starts =====
platform linux -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/kali/Desktop/新建文件夹/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.05s =====

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$ ./check.sh
3 files already formatted
All checks passed!

===== test session starts =====
platform linux -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/kali/Desktop/新建文件夹/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.05s =====

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$ echo $?
0

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q13]
└─$
```

图 2: q13 格式化、静态检查与两项单元测试均通过

结果说明

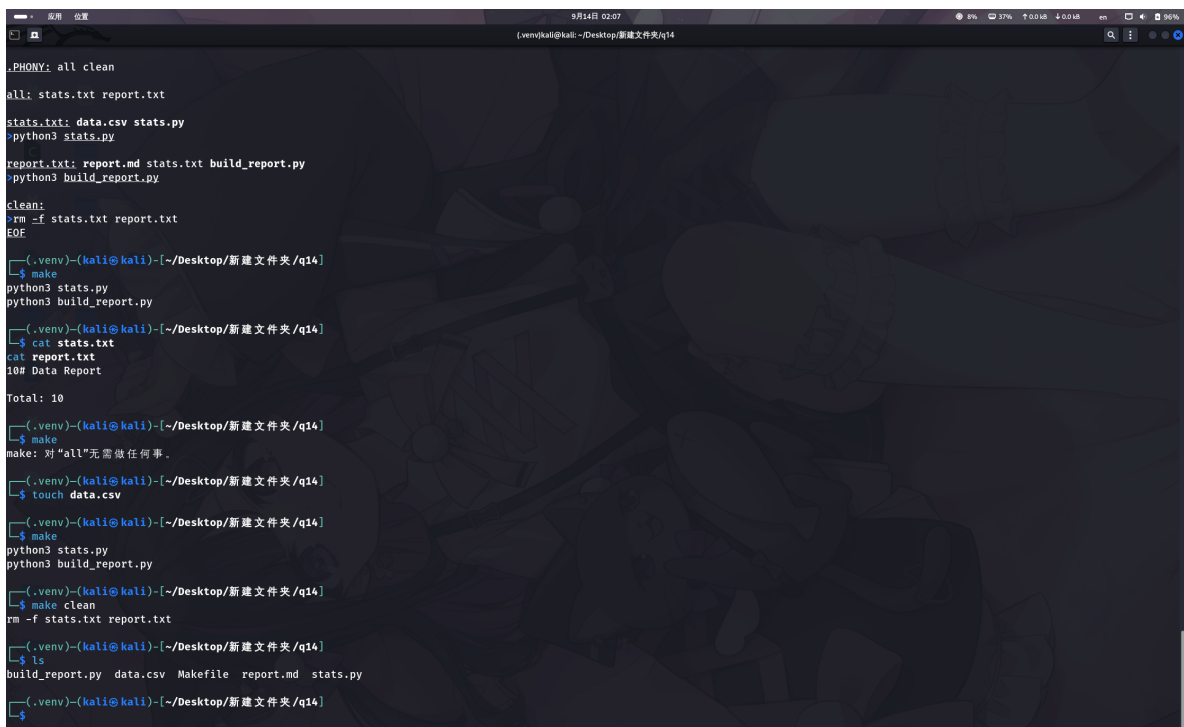
Ruff 显示 3 个文件已符合格式，静态检查通过；pytest 收集 2 项测试并全部通过，check.sh 的退出状态为 0。

练习 14 让 Make 只重建受影响的产物 (Make 依赖关系)

执行内容

stats.txt 依赖 data.csv 和 stats.py, report.txt 进一步依赖 stats.txt、report.md 与 build_report.py。首次 make 生成两个产物；随后未修改文件时不执行配方；更新 data.csv 的时间戳后，两项产物重新生成。

```
make
cat stats.txt
cat report.txt
make
touch data.csv
make
make clean
```



```
.PHONY: all clean
all: stats.txt report.txt
stats.txt: data.csv stats.py
python3 stats.py
report.txt: report.md stats.txt build_report.py
python3 build_report.py
clean:
rm -f stats.txt report.txt
EOF

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ make
python3 stats.py
python3 build_report.py

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ cat stats.txt
cat report.txt
10# Data Report

Total: 10

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ make
make: 对"all"无需做任何事。

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ touch data.csv

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ make
python3 stats.py
python3 build_report.py

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ make clean
rm -f stats.txt report.txt

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$ ls
build_report.py data.csv Makefile report.md stats.py

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q14]
└─$
```

图 3: q14 的首次构建、增量构建与 clean 验证

结果说明

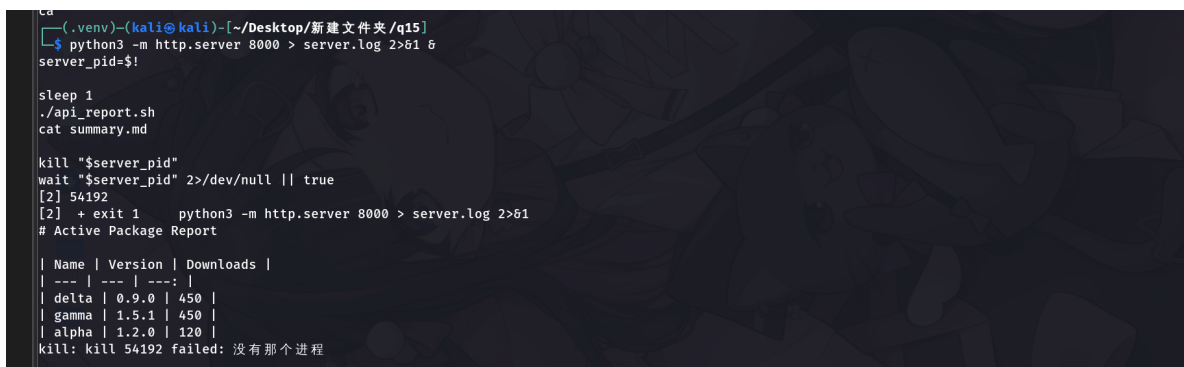
统计结果为 10，报告正确写入 Total: 10。第二次构建提示目标无需重建；touch data.csv 后两个 Python 脚本均再次执行；clean 后仅保留四个源文件和 Makefile。

练习 15 把本地 API 数据转换为可读报告 (curl 与 jq)

执行内容

在 q15 中通过 `python3 -m http.server 8000` 提供 `packages.json`, `api_report.sh` 使用 `curl` 获取 JSON, 再由 `jq` 过滤 `active` 且下载量不少于 100 的记录, 并按下载量降序、名称升序输出 Markdown 表格。

```
python3 -m http.server 8000 > server.log 2>&1 &
server_pid=$!
sleep 1
./api_report.sh
cat summary.md
kill "$server_pid"
```



```
ca
(.venv)~(kali@kali)~[~/Desktop/新建文件夹/q15]
$ python3 -m http.server 8000 > server.log 2>61 6
server_pid=$!

sleep 1
./api_report.sh
cat summary.md

kill "$server_pid"
wait "$server_pid" 2>/dev/null || true
[2] 54192
[2] + exit 1      python3 -m http.server 8000 > server.log 2>61
# Active Package Report

| Name | Version | Downloads |
| --- | --- | ---: |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
kill: kill 54192 failed: 没有那个进程
```

图 4: q15 生成的活动软件包 Markdown 报告

结果说明

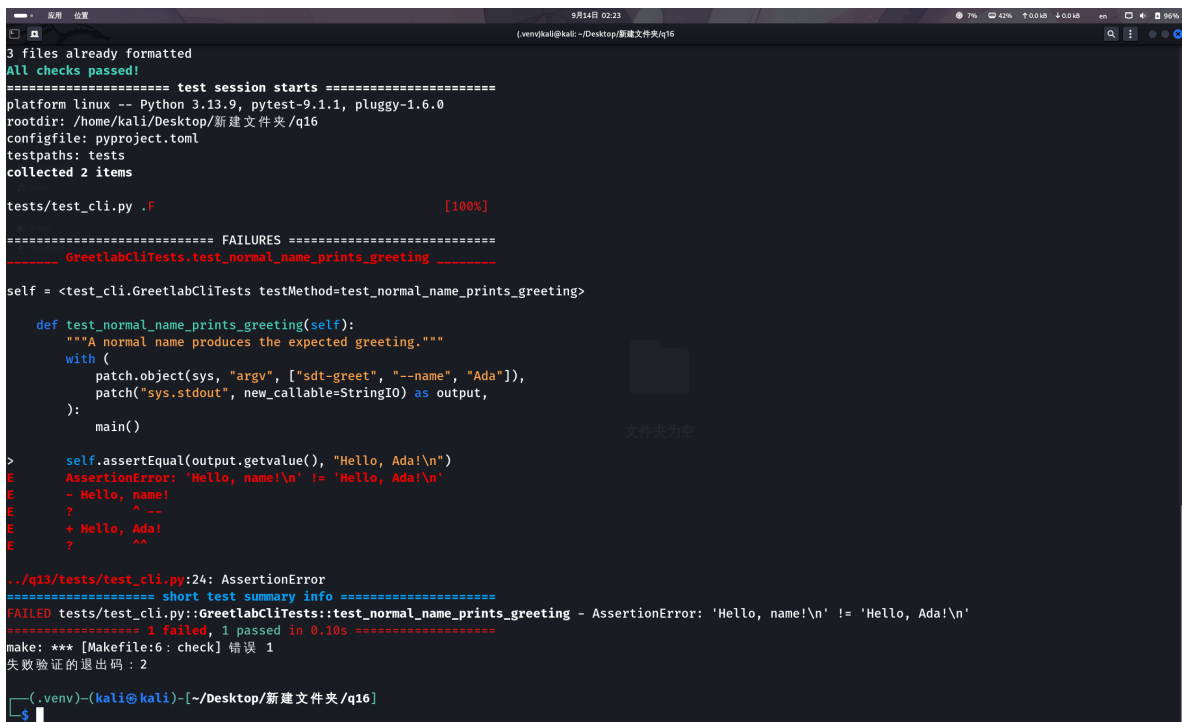
生成的表格包含 `delta`、`gamma` 和 `alpha` 三条记录, 下载量分别为 450、450、120。两个 450 下载量记录按名称升序排列, 符合排序要求。服务进程已结束再次 `kill` 的提示不影响报告生成。

练习 16 修复并交付陌生的小型工具包（测试、构建与 Git）

执行内容

q16 从 q13 复制，并初始化独立 Git 仓库。先将问候实现临时改为固定输出 Hello, name!, 再以 make check 验证正常姓名测试会失败；随后恢复使用传入姓名的实现，重新检查并构建 wheel。

```
make check          # 故意错误时失败
make check          # 修复后通过
make build
ls -lh dist/
git add .
git commit -m "feat: deliver verified greetlab package"
```

A terminal window showing the execution of a test suite. The output indicates that 3 files were already formatted and all checks passed initially. However, when running the test session, a failure occurred in the test_normal_name_prints_greeting function. The failure message states: 'Hello, name!\n' != 'Hello, Ada!\n'. The test output shows that the actual output was 'Hello, name!' while the expected output was 'Hello, Ada!'. The terminal also shows the test summary and the final error message: 'AssertionError: 'Hello, name!\n' != 'Hello, Ada!\n''.

```
3 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/kali/Desktop/新建文件夹/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .F [100%]

===== FAILURES =====
_____ GreetlabCliTests.test_normal_name_prints_greeting _____
self = <test_cli.GreetlabCliTests testMethod=test_normal_name_prints_greeting>

    def test_normal_name_prints_greeting(self):
        """A normal name produces the expected greeting."""
        with (
            patch.object(sys, "argv", ["sdt-greet", "--name", "Ada"]),
            patch("sys.stdout", new_callable=StringIO) as output,
        ):
            main()

> self.assertEqual(output.getvalue(), "Hello, Ada!\n")
E   AssertionError: 'Hello, name!\n' != 'Hello, Ada!\n'
E   - Hello, name!
E   ? ^ --
E   + Hello, Ada!
E   ? ^^

../q13/tests/test_cli.py:24: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::GreetlabCliTests::test_normal_name_prints_greeting - AssertionError: 'Hello, name!\n' != 'Hello, Ada!\n'
===== 1 failed, 1 passed in 0.10s =====
make: *** [Makefile:6: check] 错误 1
失败验证的退出码: 2

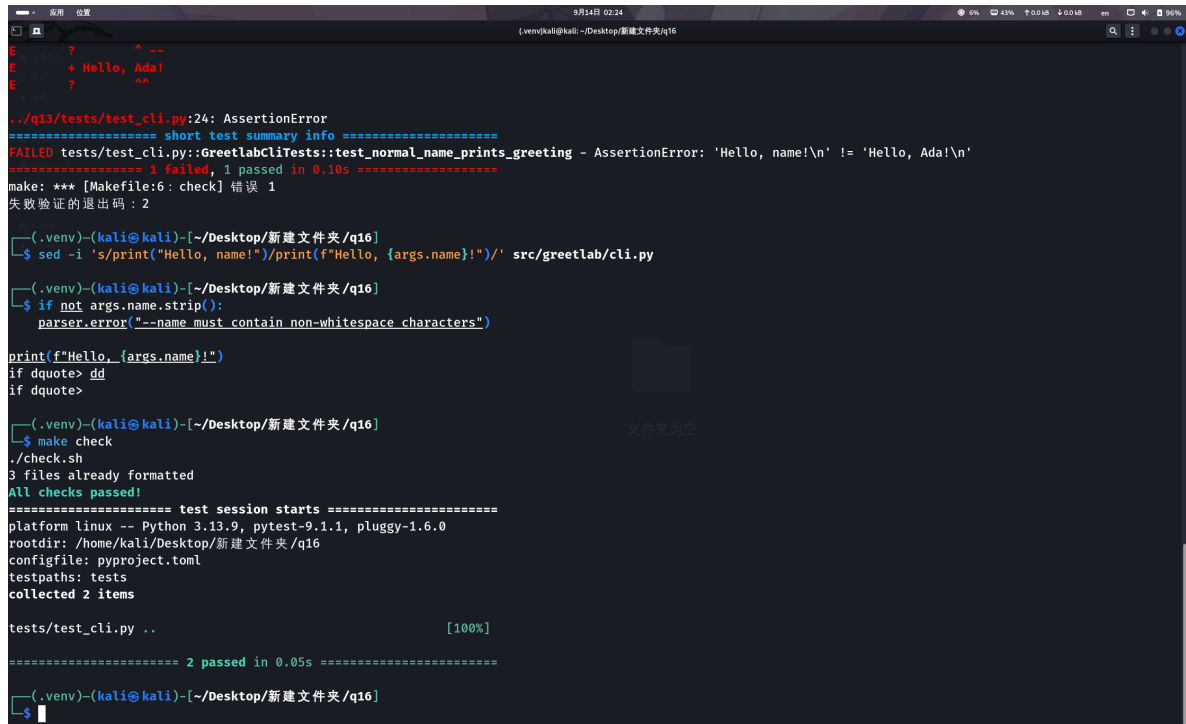
(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
```

图 5: q16 中固定问候语导致正常姓名测试失败

结果说明

失败时实际输出为 Hello, name!, 与期望的 Hello, Ada! 不一致, make check 返回非零状态, 说明测试能有效定位回归。

练习 16 修复验证与交付（测试、构建与 Git）



```

E      ?      ^ --
E      + Hello, Ada!
E      ?      ^^

../q13/tests/test_cli.py:24: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::GreetlabCliTests::test_normal_name_prints_greeting - AssertionError: 'Hello, name!\n' != 'Hello, Ada!\n'
===== 1 failed, 1 passed in 0.10s =====
make: *** [Makefile:6: check] 错误 1
失败验证的退出码: 2

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$ sed -i 's/print("Hello, name!)/print(f"Hello, {args.name}!)/' src/greetlab/cli.py

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$ if not args.name.strip():
    parser.error("--name must contain non-whitespace characters")

print(f"Hello, {args.name}!")
if dquote> dd
if dquote>

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$ make check
./check.sh
3 files already formatted
All checks passed!

===== test session starts =====
platform linux -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/kali/Desktop/新建文件夹/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                [100%]

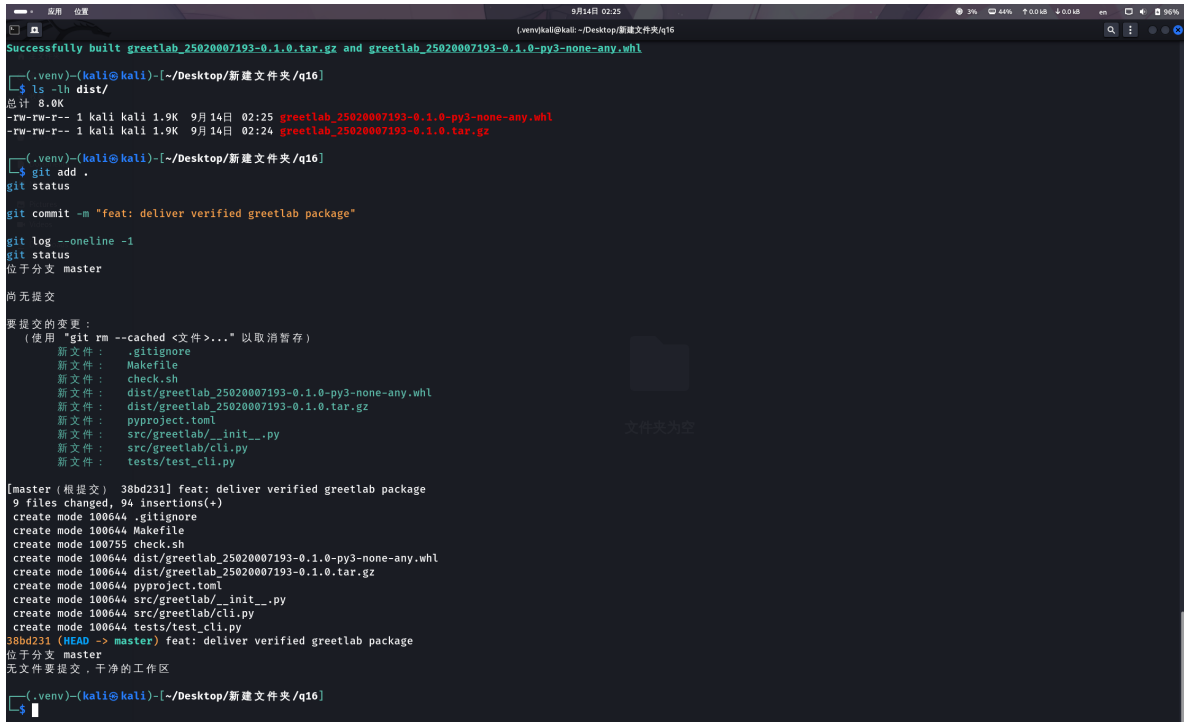
===== 2 passed in 0.05s =====

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$
```

图 6: q16 修复后 make check 通过

修复结果

恢复问候函数后，两项测试均通过，说明正常姓名与空白姓名两个行为均得到验证。



```
Successfully built greetlab_25020007193-0.1.0.tar.gz and greetlab_25020007193-0.1.0-py3-none-any.whl
(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$ ls -lh dist/
总计 8.0K
-rw-rw-r-- 1 kali kali 1.9K 9月14日 02:25 greetlab_25020007193-0.1.0-py3-none-any.whl
-rw-rw-r-- 1 kali kali 1.9K 9月14日 02:24 greetlab_25020007193-0.1.0.tar.gz

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$ git add .
git status
git commit -m "feat: deliver verified greetlab package"
git log --oneline -1
git status
位于分支 master
尚无提交

要提交的变更:
(使用 "git rm --cached <文件>..." 以取消暂存)
  新文件:   .gitignore
  新文件:   Makefile
  新文件:   check.sh
  新文件:   dist/greetlab_25020007193-0.1.0-py3-none-any.whl
  新文件:   dist/greetlab_25020007193-0.1.0.tar.gz
  新文件:   pyproject.toml
  新文件:   src/greetlab/__init__.py
  新文件:   src/greetlab/cli.py
  新文件:   tests/test_cli.py

[master (根提交) 38bd231] feat: deliver verified greetlab package
9 files changed, 94 insertions(+)
create mode 100644 .gitignore
create mode 100644 Makefile
create mode 100755 check.sh
create mode 100644 dist/greetlab_25020007193-0.1.0-py3-none-any.whl
create mode 100644 dist/greetlab_25020007193-0.1.0.tar.gz
create mode 100644 pyproject.toml
create mode 100644 src/greetlab/__init__.py
create mode 100644 src/greetlab/cli.py
create mode 100644 tests/test_cli.py
38bd231 (HEAD -> master) feat: deliver verified greetlab package
无文件要提交，干净的工作区

(.venv)-(kali@kali)-[~/Desktop/新建文件夹/q16]
└─$
```

图 7: q16 成功构建 wheel 与 source distribution，并完成 Git 提交

交付结果

make build 成功生成 wheel 和源代码压缩包；Git 提交 38bd231 后的工作区状态为干净。

练习 1 配置格式化器并验证只检查模式（格式化）

任务与执行命令

对项目中的不规范 Python 文件先运行检查，随后格式化并复查。

```
python -m ruff format --check src tests
python -m ruff format src tests
python -m ruff format --check src tests
```

结果说明

首次检查发现格式差异；格式化命令仅调整空格、引号和换行等表层风格；最终只检查模式返回 0。这符合 CI 中“检查而不修改文件”的原则。

练习 2 用 linter 捕获未使用导入 (Lint)

任务与执行命令

在示例模块故意保留 `import os`，验证静态分析后使用自动修复。

```
python -m ruff check src
python -m ruff check src --fix
python -m ruff check src
```

结果说明

Ruff 在不运行程序的前提下报告 F401（未使用的导入）。自动修复删除该导入，最终检查通过；修复后仍应人工审阅差异。

练习 3 为问候函数编写单元测试 (测试)

任务与执行命令

测试普通姓名、前后空格以及空白姓名拒绝三种行为。

```
pytest -q tests/test_greet.py  
# 3 passed
```

结果说明

测试覆盖正常输入和非法输入。空白姓名必须抛出 `ValueError`，从而防止程序悄然生成无效问候语。

练习 4 生成覆盖率报告并补足分支（测试覆盖率）

任务与执行命令

```
coverage run -m pytest
coverage report -m
coverage html
```

结果说明

补充空白姓名测试后，`greet.py` 的有效执行行均被运行。HTML 报告位于 `htmlcov/index.html`；覆盖率用于发现遗漏路径，不能替代对断言质量的判断。

练习 5 配置 pre-commit 质量门禁 (Pre-commit 钩子)

任务与执行命令

```
pre-commit install
pre-commit run --all-files
git commit -m "chore: enable quality checks"
```

结果说明

配置文件使提交前自动执行 Ruff 格式检查、Ruff lint 与 pytest。若其中任一项失败，提交会被阻止，先修复再提交。

练习 6 为 push 配置持续集成（持续集成）

任务与执行命令

```
git add .github/workflows/quality.yml
git commit -m "ci: verify format lint and tests"
git push
```

结果说明

工作流在 push 与 pull_request 时使用独立的 Ubuntu 环境安装依赖，并依次执行格式检查、lint 和测试。本地修改造成的 lint 错误应使 CI 失败。

练习 7 通过 Makefile 统一命令入口（命令运行器）

任务与执行命令

```
make format-check  
make lint  
make test  
make check
```

结果说明

开发者只需记住简短、稳定的 `make` 目标；底层工具参数集中在 Makefile 中。`check` 汇总所有质量检查，并在任一步失败时返回非零状态。

练习 8 用正则查找危险 `shell=True` 调用（正则表达式）

任务与执行命令

```
grep -RIInE 'subprocess\.Popen\([^\)]*shell[[:space:]]*=[[:space:]]*True' .  
# semgrep -l python -e 'subprocess.Popen(..., shell=True, ...)' .
```

结果说明

grep 能找出同一行内的常见写法；若参数分行、顺序变化或代码结构更复杂，纯文本模式可能漏报。语义工具 semgrep 更适合执行可维护的安全规则。

练习 9 精确替换 Markdown 项目符号 （正则查找替换）

任务与执行命令

```
python -c "import re,pathlib; p=pathlib.Path('notes.md'); s=p.read_text(); p.write_text(re.sub(r'(?m)^- (?=\S)', '* ', s))"
```

结果说明

模式锚定行首，仅替换 Markdown 列表标记 - ；日期、减号和正文中的连字符不会被误改。替换前后应使用 `git diff` 审阅。

练习 10 从 JSON 提取 name，并比较两种方案（正则与解析器）

任务与执行命令

```
python extract_name.py < person.json
python -c "import re; print(re.search(r'\"name\"\\s*:\\s*\"((?:\\\\\\\\.|[^\"])*)\", open('person.json').read()).group(1))"
```

结果说明

非贪婪或转义感知的 regex 可用于受控示例；正式 JSON 应使用 `json.load`，因为它能正确处理转义、字段顺序、嵌套结构和格式变化。

解题感悟

代码质量不是单一工具的结果，而是格式化、静态检查、测试、构建和版本控制门禁的组合。正则表达式适合明确的小范围匹配；面对结构化数据，应优先使用对应解析器。

参考资料

<https://missing-semester-cn.github.io/2026/code-quality/>